



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF SOFTWARE TECHNOLOGY AND METHODOLOGY

ELTE Informatics Chatbot Using a Small Language Model

Supervisor:

Altangerel Gereltsetseg

Assistant Professor, Ph.D

Author:

Ulziibayar Borokhul

Computer Science BSc

Budapest, 2026

Thesis Topic Registration Form

Student's Data:

Student's Name: Ulziibayar Borokhul

Student's Neptun code: PRUA5W

Educational Information:

Training programme: Computer Science BSc

I have an internal supervisor

Internal Supervisor's Name: *Altangerel Gereltsetseg*

Supervisor's Home Institution: *Department of Programming Languages and Compilers*

Address of Supervisor's Home Institution: *1117, Budapest, Pázmány Péter sétány 1/C.*

Supervisor's Position and Degree: *Assistant Professor, PHD*

Thesis Title: ELTE Informatics Chatbot Using a Small Language Model

Topic of the Thesis:

(Upon consulting with your supervisor, give a 150-300-word-long synopsis of your planned thesis.)

This thesis aims to develop a chatbot specifically designed for the ELTE Faculty of Informatics. The chatbot will act as an AI assistant that answers common administrative and academic questions, such as BSc curriculum details, prerequisite and follow-up courses, exam rules, registration, and other faculty-related information. The project will use a small open-source language model that is efficient and suitable for local deployment without high computational cost.

The work includes three main parts:

1. Collecting and preparing information from ELTE Informatics websites and documents.
2. Fine-tuning a small language model to give accurate, context-aware answers.
3. Building a simple web interface for users to interact with the chatbot.

The goal is to create a practical, privacy-safe, and low-cost chatbot that improves communication and helps students and staff quickly find relevant information.

Budapest, 2025. 10. 15.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Related Work	3
1.3	Problem Statement	4
1.4	Objective	5
1.5	Technologies Used	6
1.6	Thesis Structure	7
2	Software Requirements	8
2.1	Functional Requirements	8
2.2	Non-Functional Requirements	10
3	User Documentation	11
3.1	Overview	11
3.2	System Requirements	12
3.3	Installation	12
3.4	Using the Web Interface	15
3.5	Using the Command-Line Client	17
3.6	Example Queries	18
4	Developer Documentation	20
4.1	Design	20
4.1.1	System Architecture	20
4.1.2	Key Design Decisions	21
4.1.3	Component Design	22
4.1.4	Storage Design	24
4.2	Implementation	25
4.2.1	Project Structure	25

4.2.2	Central Configuration	27
4.2.3	Data Collection and Ingestion	29
4.2.4	Embedding and Vector Storage	32
4.2.5	Retrieval-Augmented Generation Pipeline	34
4.2.6	FastAPI Backend	37
4.2.7	Logging System	41
4.2.8	Frontend	41
5	Testing and Evaluation	44
5.1	Functional Testing	44
5.2	System Evaluation	46
5.2.1	Evaluation Setup	46
5.2.2	Results	48
5.2.3	Discussion	50
6	Conclusion	53
6.1	Summary of Contributions	53
6.2	Reflection on Goals	54
6.3	Limitations	54
6.4	Future Work	55
A	Evaluation Test Set	57
	Bibliography	63
	List of Figures	64
	List of Tables	65
	List of Algorithms	66
	List of Codes	67

Chapter 1

Introduction

1.1 Motivation

Students and staff at the Faculty of Informatics of Eötvös Loránd University face a recurring practical challenge: finding accurate answers to questions about the curriculum. Which courses must be completed before enrolling in a given subject? What are the rules for repeating a failed exam? The answers exist — scattered across faculty web pages, PDF regulations, and DOCX course descriptions — but locating the right document and extracting a clear answer takes considerably longer than it should.

General-purpose search engines return ranked lists of pages; they do not synthesise an answer from multiple sources or understand the specific terminology of Hungarian higher education. As documentation grows each semester and more information migrates to hard-to-find sub-pages, the gap between the information that exists and what students can efficiently find continues to widen.

1.2 Related Work

Several universities have built AI-powered platforms that demonstrate the viability of language models in higher education. Examining them critically, however, reveals a consistent set of design assumptions that make them fundamentally different from the system developed in this thesis.

UM-GPT (University of Michigan) is a university-wide platform on Microsoft Azure offering access to multiple large language models, with a companion RAG

tool, Maizey, that lets any user build a chatbot from a custom dataset [1]. While Maizey shares the same retrieval mechanism as this system, it is general-purpose: users define their own knowledge base and purpose, rather than targeting a specific institutional need.

ZotGPT (UC Irvine) includes ClassChat, which allows instructors to configure an AI assistant around their course materials [2]. Its orientation is pedagogical — it supports teaching within a classroom context and does not address the broader institutional information problem that students navigate independently.

CreateAI (Arizona State University) aggregates over 50 AI models into a single platform for building custom applications across the entire university [3]. Its breadth is also its limitation: a platform spanning dozens of unrelated domains is not optimised for precise retrieval over a single faculty’s document corpus.

A common profile emerges across all three: cloud infrastructure, general-purpose scope, and significant IT investment. None target a fixed domain-specific corpus, and none were evaluated on whether they correctly answer the institutional questions students actually ask. This is the gap the present work addresses.

1.3 Problem Statement

The related systems above establish that LLM-powered question answering is viable in higher education, but they do not provide a usable template for the ELTE Faculty of Informatics. Two concrete problems block direct adoption.

Domain knowledge and hallucination. General-purpose LLMs have no knowledge of ELTE-specific regulations, course codes, or administrative procedures. Without grounding in official faculty documents, a language model will hallucinate plausible-sounding but incorrect answers — a particularly harmful failure mode when a wrong answer about prerequisites or exam rules has real consequences for a student’s progress.

Resource and infrastructure constraints. Deploying a cloud-hosted platform requires dedicated IT teams and substantial recurring costs that a single faculty cannot sustain. Any practical solution must run on commodity hardware with no GPU and no cloud subscription.

No dedicated question-answering system currently exists for the ELTE Faculty of Informatics. Students rely on web search, email queries to administrative offices, or peer advice — all slow, inconsistent, and unscalable.

Approach: retrieval-augmented generation over fine-tuning. The original project specification proposed fine-tuning a small language model on faculty documents. During the design phase this approach was set aside for three reasons. First, fine-tuning requires labelled question-answer training pairs, not raw documents; constructing a dataset of sufficient quality and breadth would itself have been a substantial project. Second, a fine-tuned model is static: every time a regulation changes or a new course is added, the model must be retrained from scratch. Third, factual recall in fine-tuned sub-4B models is unreliable — small models trained on small corpora tend to hallucinate rather than reproduce memorised facts accurately. Retrieval-augmented generation addresses all three problems: it requires no training data, updates by re-indexing rather than retraining, and grounds each answer in a retrieved passage rather than relying on parametric memory. All goals stated in the original specification — local deployment, privacy, low cost, and accurate faculty-specific answers — are preserved under this design.

1.4 Objective

The result of this work is a web-based chatbot for the ELTE Faculty of Informatics. It crawls the faculty website, indexes the collected documents into a local vector database, and answers student questions in natural language by retrieving relevant passages and generating grounded responses through a locally running language model. The entire system runs on a standard laptop with no GPU and no cloud dependency.

This thesis designs, implements, and evaluates a locally deployed question-answering system for the ELTE Faculty of Informatics, built on retrieval-augmented generation. The goal is to provide accurate, document-grounded answers to student questions without the infrastructure investment or general-purpose scope of the platforms examined above. The specific goals are:

- Collect and index publicly available faculty documents — web pages, PDFs, and course descriptions — into a searchable local knowledge base.

- Answer natural language questions by retrieving relevant document passages and generating grounded responses, rather than relying on the general knowledge of a language model.
- Run entirely on commodity hardware without a GPU or cloud subscription.
- Provide a browser-based chat interface with source references so users can verify and follow up on every answer.
- Evaluate the system empirically against a no-RAG baseline to measure whether retrieval meaningfully improves answer accuracy.
- Deploy as a ready-to-run web service, so that moving to a faculty server requires infrastructure provisioning only, with no code changes.

1.5 Technologies Used

Two hard constraints shaped every technology decision: the system must run on commodity hardware with 8 GB of RAM and no discrete GPU, and no query or document may leave the local machine. The following components form the complete stack.

- **Ollama** serves large language models locally over a REST interface, handling quantisation and memory mapping automatically. Switching models requires only a one-line configuration change.
- **Llama 3.2 (3B)** and **Gemma 3 (4B)** are the two open-weight models evaluated in this thesis. Both run at 4-bit quantisation, reducing their memory footprint to approximately 2–2.5 GB and making CPU-only inference feasible.
- **all-MiniLM-L6-v2** encodes document chunks and user queries into 384-dimensional vectors for semantic similarity search, running efficiently on CPU.
- **ChromaDB** stores chunk embeddings and answers nearest-neighbour queries at runtime. It runs in-process with no separate server and persists to a plain directory on disk.

- **FastAPI** exposes the RAG pipeline as a REST API and serves the browser interface as static files, chosen for its low boilerplate and built-in request validation.
- **SQLite** logs every interaction — query, answer, sources, and response time — to a single file on disk with no server process or configuration required.
- **Vanilla JavaScript** powers the browser interface as a single self-contained HTML file. Avoiding a frontend framework eliminates a build step and all Node.js dependencies entirely.
- **Docker** packages the entire stack into isolated containers, so that moving from a local machine to a faculty server requires infrastructure provisioning only, with no code changes.

1.6 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 defines the system requirements. It states the functional requirements covering the capabilities exposed to the user, and the non-functional constraints including resource limits, latency, privacy, and offline operation.

Chapter 3 provides user documentation: a use-case diagram, step-by-step instructions for the web interface and command-line client, and annotated example queries.

Chapter 4 covers developer documentation. It describes the system architecture and component design, then details the full implementation across the data pipeline, embedding, RAG backend, REST API, logging, frontend, and deployment.

Chapter 5 presents the testing and evaluation of the system. It covers functional testing, the empirical RAG vs. baseline evaluation, and a user evaluation.

The Conclusion summarises the findings, reflects on the key design decisions, and outlines directions for future work.

Chapter 2

Software Requirements

This chapter defines what the ELTE IK Assistant must do and under what constraints it must operate. The functional requirements describe the capabilities the system exposes to the user. The non-functional requirements define the resource, privacy, latency, and reliability constraints that any correct implementation must satisfy. Interaction logging and the offline data pipeline are internal system concerns and do not appear as user-facing requirements.

2.1 Functional Requirements

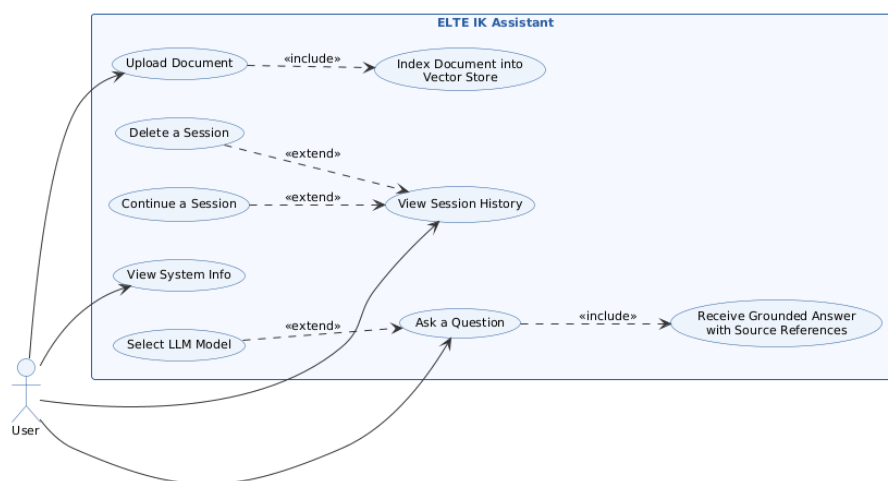


Figure 2.1: Use case diagram for the ELTE IK Assistant

Figure 2.1 illustrates the primary interactions between the user and the system. The single actor is the **User**, representing any student or staff member accessing the assistant through the browser interface. Asking a question is the central use case: it

always includes receiving a grounded answer with source references, and optionally extends to selecting a language model before submitting. Document upload always includes indexing the uploaded file into the vector store. Viewing session history optionally extends to continuing or deleting an existing session.

R-F-1 Natural language querying: the system accepts a free-text English question and returns a textual answer with source citations. When the answer is absent from the indexed documents, the system must say so explicitly rather than generating an unsupported response.

R-F-2 Model selection: the user may select which language model handles the current query. The system must apply the selected model to all subsequent queries in the session without requiring a restart.

R-F-3 Document upload: the user may upload a document that is immediately chunked, embedded, and added to the vector store, making its contents available for retrieval in the same session.

R-F-4 Embedding and retrieval: each chunk is encoded into a dense vector and stored in a local persistent vector database. At query time the top- K most similar chunks are retrieved; K is configurable.

R-F-5 Session history: the system persists conversation history across page reloads. The user may view past sessions, continue a previous session, or delete a session entirely.

R-F-6 System information: the user may view the current system status, including model availability and vector store state, without submitting a query.

R-F-7 REST API: the backend exposes a `/health` endpoint reporting system status and a `/chat` endpoint that accepts a message and returns the answer with source references. Both endpoints must be usable independently of the browser interface.

R-F-8 Web interface: a browser-based chat UI displays conversation history, source file names, model availability, and inline error messages without requiring any installation on the client side.

2.2 Non-Functional Requirements

R-NF-1 Privacy: no query, chunk, or answer may leave the local machine. The system must remain fully functional in a network-isolated environment after initial setup.

R-NF-2 Resource constraints: the system must run on 8 GB RAM with no discrete GPU, using a quantised (4-bit or 8-bit) language model and a CPU-friendly embedding model.

R-NF-3 Response latency: end-to-end response time must not exceed 90 seconds on the reference hardware. Embedding and retrieval must complete in under one second; generation typically takes 30–80 seconds on CPU.

R-NF-4 Offline operation: after the initial model pull and corpus crawl, no network connection is required. All subsequent pipeline steps run entirely from local storage.

R-NF-5 Reliability: if the language model process is unavailable, `/health` must report it and `/chat` must return an HTTP 503. The vector store and log database must not be corrupted on unexpected shutdown, and re-running the pipeline must be idempotent.

R-NF-6 Configurability: all tunable parameters — model names, K , temperature, paths, timeouts — are defined in a single configuration module. Swapping models requires only a one-line change there; changing the embedding model additionally requires rebuilding the vector store.

R-NF-7 Deployability: the system must be deployable on a faculty server without modification to the application code. All components are containerised with Docker, so that moving from a local prototype to a live university service requires infrastructure provisioning only — configuring ports, paths, and model names — with no changes to any other part of the codebase.

Chapter 3

User Documentation

This chapter describes how to set up and use the ELTE IK Assistant. Two setup paths are provided: a Docker-based path and a manual path. Choose based on your situation — Docker is better for isolated or shared deployments, the manual path is simpler for a single machine.

3.1 Overview

The ELTE IK Assistant is a question-answering chatbot for the Faculty of Informatics at Eötvös Loránd University. It answers natural language questions about the curriculum, course requirements, exam rules, and administrative procedures, drawing its answers exclusively from official faculty documents.

The system works in two stages. Before first use, a data pipeline crawls the faculty website and indexes the collected documents into a local vector database. At query time, the assistant retrieves the most relevant document passages and passes them to a locally running language model, which generates a grounded answer with source references. All processing happens on the local machine — no data is sent to any external service.

The key components are **Ollama** (local language model server), **Llama 3.2 (3B)** and **Gemma 3 (4B)** (the supported language models), **ChromaDB** (vector database), and **FastAPI** (web server). No cloud account or GPU is required. (see Section ?? for more details)

3.2 System Requirements

Table 3.1 lists the minimum system requirements for running the ELTE IK Assistant.

Component	Requirement
Operating system	Windows 10/11, macOS 12+, or Ubuntu 22.04+
RAM	8 GB minimum (16 GB recommended)
Disk space	≈6 GB (models + vector store + crawled data)
Internet access	Required during first-run setup only

Table 3.1: Minimum system requirements.

3.3 Installation

Option A — Docker

Docker Compose orchestrates all services automatically: the Ollama language model server, both model downloads, the knowledge base build, and the FastAPI application. This option is best suited for isolated or shared deployments.

First-run note: on the first `docker compose up`, the container crawls ELTE documents and builds the vector index before starting the server. This takes approximately 5–10 minutes depending on internet speed and hardware. Subsequent starts are immediate.

1. Install **Docker Desktop** (includes Docker Compose) from the official Docker website for your operating system.
2. Clone or download the project repository:

```
1 git clone https://github.com/borohull/Thesis.git
2 cd Thesis/elte_chat
```

3. Start all services:

```
1 docker compose up -d
```

4. Monitor first-run progress:

```
1 docker compose logs -f app
```

Wait until `Application startup complete` appears in the logs.

5. Open the chat interface at `http://localhost:8000/static/index.html`.
6. To stop the containers:

```
1 docker compose down
```

Option B — Manual Setup

This option is best suited for running the assistant on a single local machine. Steps 1–4 are performed **once** by whoever sets up the system. End users only need step 5 (opening the browser).

Prerequisite: Python 3.10 or newer must be installed on your system.

1. Install **Ollama** from <https://ollama.com/download>, then start the service and pull both language models:

```
1 ollama serve
2 ollama pull llama3.2:3b
3 ollama pull gemma3:4b
```

2. Clone or download the project repository:

```
1 git clone https://github.com/borohull/Thesis.git
2 cd Thesis/elte_chat
```

3. Run the setup script. This creates a virtual environment, installs dependencies, crawls ELTE documents, and builds the vector index.

macOS / Linux:

```
1 chmod +x setup.sh
2 ./setup.sh
```

Windows:

```
1 setup.bat
```

The script prints progress at each stage. A complete first run takes approximately 5–10 minutes. Listing 3.2 shows the expected terminal output.

```
1 === ELTE IK Assistant Setup ===
2 [1/6] Starting Ollama service...
```

```
3 [2/6] Pulling language models (this may take several minutes)
   ...
4 pulling manifest
5 pulling dde5aa3fc5ff: 100% 2.0 GB
6 ...
7 success
8 pulling manifest
9 pulling aeda25e63ebd: 100% 3.3 GB
10 ...
11 success
12 [3/6] Setting up Python environment...
13 [4/6] Crawling ELTE documents (this will take several minutes)
   ...
14 [SAVE] https://inf.elte.hu/en -> index.html
15 [SAVE] https://inf.elte.hu/en/courses -> courses.html
16 [PDF-SAVE] https://inf.elte.hu/... -> regulations.pdf
17 [SAVE] https://www.elte.hu/en/quaestura-office -> quaestura
   -office.html
18 ...
19 Crawl complete -- saved: 482, skipped: 0, failed: 5, PDFs: 754,
   DOCXs: 46
20 [5/6] Building knowledge base (chunking + embedding)...
21 === build_index.py: ELTE IK knowledge base builder ===
22 --- Phase 1: Ingestion ---
23 Scan: 482 files on disk, 0 in manifest
24 new:          482
25 updated:      0
26 deleted:      0
27 unchanged:    0
28 --- Phase 2: Embedding ---
29 Loaded 6128 chunks
30 Batches: 100% 192/192
31 Upserted 6128 chunks into 'elte_ik'
32 Collection now has 6128 documents
33 === Done. Knowledge base is up to date. ===
34 [6/6] Setup complete.
35
36 To start the server:
37     source .venv/bin/activate && uvicorn app.main:app (macOS/
   Linux)
38     .venv\Scripts\activate && uvicorn app.main:app (Windows)
```



```
39  
40 Then open: http://localhost:8000/static/index.html
```

Code 3.1: Expected terminal output during first-run setup.

On subsequent runs the crawler skips already-downloaded files and the embedding step skips unchanged chunks, so re-running setup is fast and safe.

4. Start the server:

```
1 # macOS / Linux  
2 source .venv/bin/activate && uvicorn app.main:app  
3  
4 # Windows  
5 .venv\Scripts\activate && uvicorn app.main:app
```

Code 3.2: Expected terminal output during first-run setup.

5. Open the chat interface at `http://localhost:8000/static/index.html`.

To verify the server is running correctly, run:

```
1 curl http://localhost:8000/health
```

A successful response is `{"status": "ok", "ollama": true}`. If `"ollama": false` appears, Ollama is not running or the selected model has not been pulled.

3.4 Using the Web Interface

Open `http://localhost:8000/static/index.html` in any modern browser once the server is running. The interface has three main areas:

- **Left sidebar** — lists all chat sessions; click one to resume it or use **New chat** to start a fresh conversation.
- **Central panel** — the main chat area where questions are typed and answers are displayed.
- **Right sidebar** — lists all available Ollama models, shows connectivity status of Backend API and Ollama, displays knowledge base size and uploaded documents. If the Backend API is offline, the interface will display an error message and the chat functionality will be disabled.

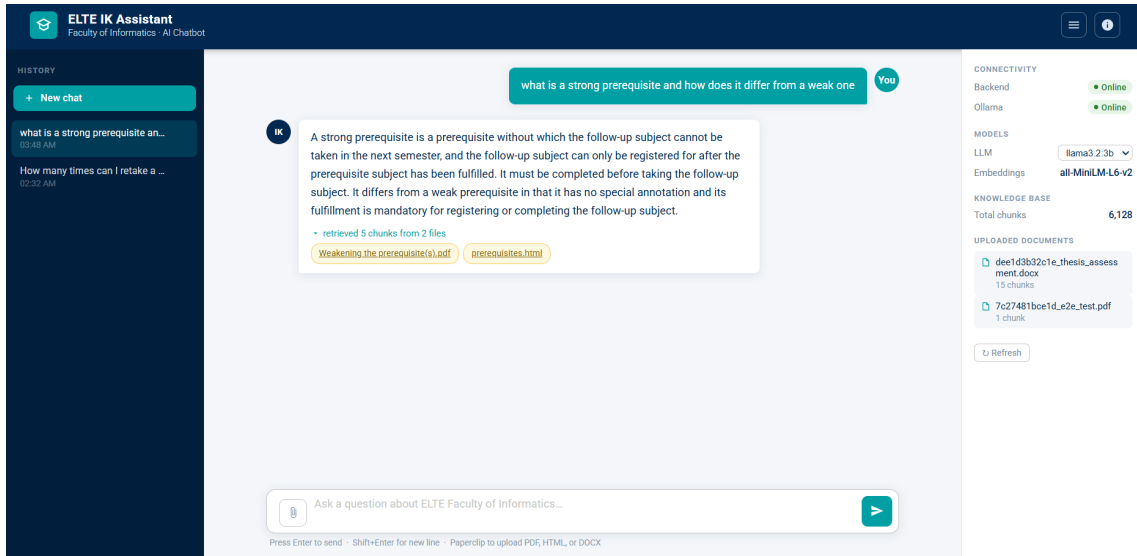


Figure 3.1: The ELTE IK Assistant web interface.

Asking a question. Type a question in the input field at the bottom of the chat panel and press **Enter** or click the send button. The assistant retrieves relevant document chunks, constructs a grounded prompt, and returns an answer along with the source file names it used. Questions should relate to the ELTE Faculty of Informatics — curriculum, regulations, procedures, and student services. On a CPU-only machine with 8 GB of RAM, typical response times are 30–80 seconds per query. The first query after startup may be slower while the model loads into memory.

Source references. Every answer displays the names of the source documents used to construct it (for example `elte_ik_regulations.html` or `TAJ.pdf`). These are the actual files retrieved from the knowledge base. If an answer looks incorrect or incomplete, the source names indicate which documents to check directly.

Selecting a language model. The right sidebar lists all Ollama models available on the machine. Click a model name to switch for the current session. `llama3.2:3b` is the default and works on most hardware. `gemma3:4b` is also available and may produce different quality results.

Managing chat sessions. Each conversation is stored as a named session. Sessions appear in the left sidebar and persist across browser refreshes and server restarts. Click **New chat** to start a new conversation, or hover over an existing session and click the $\times$ button to delete it.

Uploading custom documents. The knowledge base can be extended with your own files. Click **Upload document** above the input field, select a PDF, HTML, or DOCX file (maximum 20 MB), and wait for the confirmation message. The document is chunked, embedded, and available for retrieval immediately — no server restart required.

3.5 Using the Command-Line Client

A command-line client is provided as an alternative to the web interface, suitable for scripting and testing. The FastAPI server must be running before starting the client:

```
1 python scripts/chat_cli.py
```

On startup the client displays a banner and connects to the server. Figure 3.2 shows the interface.

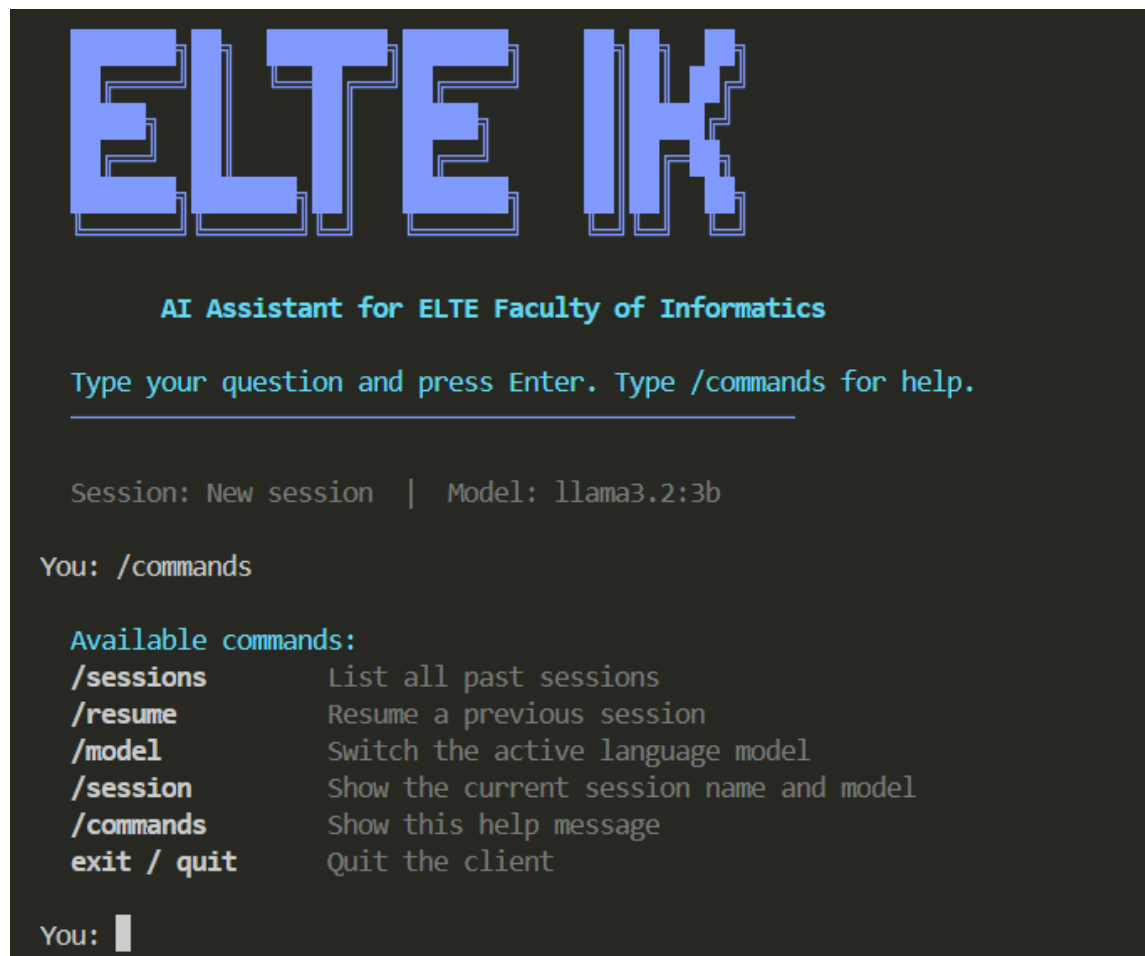


Figure 3.2: The ELTE IK Assistant command-line client.

Type a question at the `You:` prompt and press Enter. Each answer is followed by the source documents used to construct it. The client supports the following commands:

- `/commands` — display the full list of available commands.
- `/sessions` — list all past conversation sessions.
- `/resume` — select and resume a previous session.
- `/model` — switch the active language model for the current session.
- `/session` — display the current session name and active model.
- `exit` / `quit` / `Ctrl+C` — quit the client.

Each new run starts a fresh session. Conversations persist on the server and can be resumed in a later run using `/resume`.

3.6 Example Queries

The following examples illustrate the range of questions the system handles and how it responds to out-of-scope input.

Example 1 — Prerequisite query. *Question:* “What is a strong prerequisite and how does it differ from a weak one?”

Answer: A strong prerequisite is a prerequisite without which the follow-up subject cannot be taken in the next semester, and the follow-up subject can only be registered for after the prerequisite subject has been fulfilled. It must be completed before taking the follow-up subject. It differs from a weak prerequisite in that it has no special annotation and its fulfillment is mandatory for registering or completing the follow-up subject.

Sources: Weakening the prerequisite(s).pdf

Example 2 — Exam rules. *Question:* “How many times can I retake a failed exam in one semester?”

Answer: A student may attempt an exam up to three times per exam period. If all three attempts are failed, the course must be re-registered in a subsequent semester.

Sources: ELTE Academic Regulations.pdf

Example 3 — Registration procedure. *Question:* “How do I see my ranking when registering for a subject”

Answer: To see your ranking when registering for a subject, follow these steps: 1. Find the subject you want to register for. 2. Click the arrow icon at the end of the corresponding row. 3. Then click "Details" to view your score and your position in the ranking. This will show you your ranking sequence and lucky point (ranking score 1 and 2, respectively).

Sources: registration for subjects BSc_MSc (1).pdf

Example 4 — Out-of-scope query. *Question:* “What is the capital city of Australia?”

Answer: The information needed to answer this question is not available in the documents I have access to. I can only answer questions related to the ELTE Faculty of Informatics curriculum, regulations, and procedures.

Sources: (none)

The fourth example demonstrates the system’s refusal behaviour: when no relevant chunks are retrieved, the assistant explicitly acknowledges the gap rather than generating an unsupported answer.

Chapter 4

Developer Documentation

This chapter covers the full developer documentation for the ELTE IK Assistant. It is divided into two parts: the design section describes the system architecture, key design decisions, component structure, and storage model; the implementation section details how each layer was built.

4.1 Design

This section describes the architecture of the ELTE IK Assistant, the reasoning behind its key design decisions, the module structure of the application, and the storage model used to persist data.

4.1.1 System Architecture

The ELTE IK Assistant is organised as three layers:

1. **Data pipeline** — a standalone crawler script downloads raw documents from the ELTE websites; a separate ingestion and embedding pipeline cleans, chunks, and encodes those documents into vector embeddings, then stores them in a local vector database. This pipeline is run once by the system administrator and re-run whenever the source documents change.
2. **Online serving layer** — a FastAPI web server that accepts a user's natural-language question, retrieves the most relevant document chunks from the vector database, builds a prompt, calls a locally running language model, and returns the generated answer together with source references.

3. **Presentation layer** — a single-page browser UI that maintains session state, renders the conversation, and displays source file metadata alongside each answer.

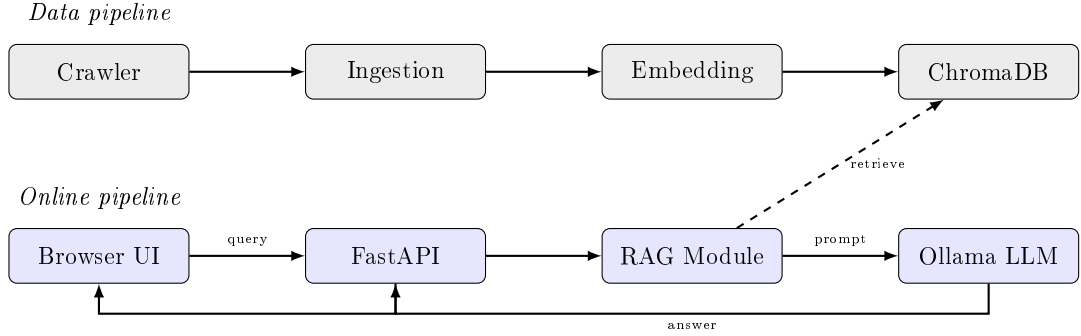


Figure 4.1: High-level architecture of the ELTE IK Assistant.

Figure 4.1 illustrates the high-level data flow between these layers. The data pipeline (top) is run once to build the vector index; the online pipeline (bottom) handles every user query at runtime.

The strict separation between the data and online pipelines is a deliberate consequence of the offline-operation requirement (R-NF-1): once the vector index is built, the serving layer requires no network access and its only disk reads are ChromaDB queries and SQLite log operations.

4.1.2 Key Design Decisions

Local LLM via Ollama. Running the language model locally satisfies both the privacy requirement — no query or document ever leaves the machine — and the offline requirement. The cost is higher latency: on the development hardware, median response times range from 25 to 180 seconds depending on the model. This trade-off was unavoidable given the constraints.

Model selection. `llama3.2:3b` and `gemma3:4b` were selected as the leading instruction-tuned open-weight models in the sub-5B parameter range available through Ollama. The 8 GB RAM constraint rules out 7B+ models at 16-bit precision (≈ 14 GB); both evaluated models use 4-bit quantisation, reducing their footprint to approximately 2 GB and 2.5 GB respectively, leaving sufficient headroom for the embedding model, ChromaDB index, and operating system. Both models also sup-

port instruction-following out of the box, which is necessary for the grounded-answer prompt to work reliably.

Chunking strategy. Documents are split into 800-character chunks with a 150-character overlap. This size is large enough to contain a complete regulation clause yet small enough for accurate embedding. The overlap ensures no sentence is silently dropped at a split boundary.

Top- k retrieval. Evaluation was conducted at $k = 5$; the production default was set to $k = 3$ to reduce inference latency on CPU-only hardware, with minimal impact on answer quality. At 800 characters per chunk, three chunks contribute approximately 2400 characters of context — well within the context window of both evaluated models.

Temperature. A temperature of 0.1 makes the model near-deterministic, which is appropriate for a factual assistant. Exactly 0 was avoided due to degenerate repetition loops observed during development.

Containerisation with Docker. Docker Compose packages the entire stack into isolated containers, directly satisfying the deployability requirement: moving to a faculty server requires only infrastructure provisioning with no code changes. The manual setup path remains available for single-machine users.

4.1.3 Component Design

The application is composed of five Python modules in the `app/` package, each with a single responsibility, plus two data pipeline scripts that run outside the serving layer. All modules read their tunable parameters from a shared configuration module (`app/settings.py`); this dependency is not shown in Figure 4.2 to avoid cluttering the arrows, but it applies to every module listed below.

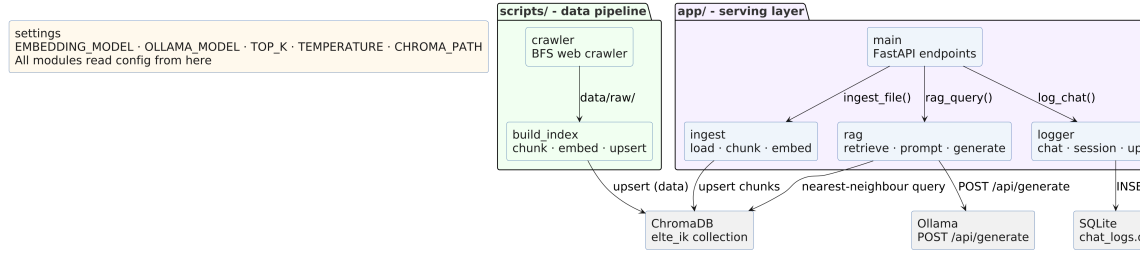


Figure 4.2: Module structure and dependencies of the ELTE IK Assistant.

Data pipeline scripts. Two scripts build the knowledge base and are never imported by the serving layer at runtime.

- **Crawler** (`scripts/crawler.py`) — breadth-first crawl of the ELTE websites; downloads HTML, PDF, and DOCX files into `data/raw/` and maintains a SHA-256 manifest for resumable runs (see Section 4.2.3). Re-running the crawler is safe: unchanged files are skipped automatically.
- **Build index** (`scripts/build_index.py`) reads the raw files produced by the crawler, applies format-specific loaders to extract clean text, splits the text into overlapping chunks, encodes each chunk with `all-MiniLM-L6-v2` (see Section 4.2.4), and upserts the vectors into ChromaDB incrementally. Only new or changed files are re-processed on each run.

Serving layer modules. The following five modules are loaded when the FastAPI process starts and handle all runtime activity.

- **settings** (`app/settings.py`) — all tunable constants in one place; no business logic. Every other module imports from it, so a single edit propagates everywhere.
- **rag** (`app/rag.py`) — core pipeline: encodes the query, retrieves top- k chunks, builds the prompt, calls Ollama, and returns the answer with source references. The embedding model and ChromaDB client are held as singletons, loaded once at startup.
- **ingest** (`app/ingest.py`) — chunks and embeds user-uploaded files directly into the live ChromaDB collection, making them available for retrieval immediately without a server restart.
- **logger** (`app/logger.py`) — owns the SQLite database; the only module that reads from or writes to it. Records chat interactions, sessions, and uploads.

- **main** (`app/main.py`) — FastAPI entry point; wires HTTP endpoints to the other modules and mounts the static frontend. It contains no business logic of its own.

4.1.4 Storage Design

The system uses two runtime stores, accessed on every request, and two build artifacts produced by the data pipeline and consumed during index construction. All four are plain files on the local filesystem — no database server, no cloud storage, and no network filesystem are involved.

Store	Location	Contents and access pattern
ChromaDB	<code>data/processed/chroma_db/</code>	6112 chunk embeddings (384-dim float32) and metadata. Written once by the data pipeline; read on every query via approximate nearest-neighbour search.
SQLite	<code>data/logs/chat_logs.db</code>	One row per chat turn: session id, user message, assistant answer, sources, response time, and timestamp. Appended on every request; queried for session listing and history retrieval.
<code>chunks.json</code>	<code>data/processed/chunks.json</code>	All document chunks with text and metadata produced by the ingestion step. Written by <code>build_index.py</code> after chunking; read by the embedding step to populate ChromaDB. Not accessed at runtime.
<code>manifest.json</code>	<code>data/processed/manifest.json</code>	SHA-256 digests, chunk counts, and timestamps for every crawled file. Written and read by the data pipeline to detect new, updated, and deleted files during incremental re-indexing. Not accessed at runtime.

Table 4.1: Runtime stores (above double line) and data pipeline artifacts (below double line) used by the ELTE IK Assistant.

Because the entire system state is contained in these four files, the knowledge base can be backed up or migrated by copying the `data/` directory, and the serving

layer can be restarted on any machine that has the directory without rebuilding the index.

4.2 Implementation

The implementation follows two phases. The data pipeline — collection, processing, and embedding — runs once offline to build the knowledge base. The online serving layer — retrieval, answer generation, and the HTTP API — runs on every user query. The subsections below follow this order, starting with project structure and configuration before covering each phase in turn.

All development and testing was performed on the machine specified in Table 4.2, which matches the target hardware profile defined in the resource constraint (R-NF-2).

Component	Specification
System	Acer Aspire A715-75G
CPU	Intel Core i5-10300H (8 CPUs, 2.5 GHz)
RAM	8 GB DDR4
GPU	NVIDIA GeForce GTX 1650 (Integrated)
OS	Windows 11 Enterprise 64-bit (Build 26200)
Python	3.14.3

Table 4.2: Development and test machine specifications.

4.2.1 Project Structure

The repository is organised into clearly separated concerns, as summarised in Table 4.3. The data pipeline scripts are never imported by the serving layer; the `app/` package contains everything loaded at runtime.

Path	Purpose
<i>Setup and deployment</i>	
<code>setup.sh</code> / <code>setup.bat</code>	First-run setup scripts for macOS/Linux and Windows. Pull Ollama models, create the virtual environment, run the crawler, and build the index.

Path	Purpose
<code>docker-compose.yml</code>	Orchestrates the Ollama service, model pull, and FastAPI application as isolated containers for server deployment.
<i>Data pipeline (run once; not imported at runtime)</i>	
<code>scripts/crawler.py</code>	Resumable BFS web crawler for both ELTE domains. Downloads HTML, PDF, and DOCX files into <code>data/raw/</code> .
<code>scripts/build_index.py</code>	Production pipeline script. Ingests raw files, chunks them, generates embeddings with <code>all-MiniLM-L6-v2</code> , and upserts vectors into ChromaDB incrementally using the SHA-256 manifest.
<i>Serving layer (app/ package, loaded at runtime)</i>	
<code>app/settings.py</code>	Centralised configuration constants: paths, model names, retrieval parameters, and timeouts.
<code>app/rag.py</code>	Retrieval, prompt construction, and Ollama integration. Holds the ChromaDB client and embedding model as module-level singletons.
<code>app/ingest.py</code>	File loading, chunking, and live embedding for user-uploaded documents.
<code>app/logger.py</code>	SQLite-backed logging for chat interactions, sessions, and uploads.
<code>app/main.py</code>	FastAPI application entry point. Wires all HTTP endpoints; contains no business logic.
<code>app/static/</code>	Single-page HTML/CSS/JS frontend, served as a static file.
<i>Development and evaluation (not part of the deployed system)</i>	
<code>scripts/chat_cli.py</code>	Interactive terminal client for the chat API.
<code>tests/</code>	End-to-end pytest suite covering all API endpoints.

Path	Purpose
notebooks/01-03	Exploration notebooks for ingestion, embedding, and RAG pipeline development. Superseded by <code>build_index.py</code> for production use.
notebooks/04_evaluation.ipynb	Evaluation of retrieval and answer quality across four model/RAG configurations.
<i>Data directories (not tracked in version control)</i>	
data/raw/	Crawled source documents (HTML; PDFs in <code>linked_pdfs/</code> ; DOCX in <code>linked_docx/</code>).
data/processed/	<code>chunks.json</code> , <code>manifest.json</code> , <code>pending_changes.json</code> , and the ChromaDB vector store.
data/logs/	SQLite chat log database.
data/uploads/	User-uploaded documents stored after ingestion.

Table 4.3: Project directory structure and file responsibilities.

4.2.2 Central Configuration

All tunable parameters are centralised in `app/settings.py` so that they can be changed in one place without touching any business logic. Every other module imports from it; no constants are defined elsewhere.

```

1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).parent.parent
4 CHROMA_PATH = str(BASE_DIR / "data/processed/chroma_db")
5 CHUNKS_PATH = str(BASE_DIR / "data/processed/chunks.json")
6 MANIFEST_PATH = str(BASE_DIR / "data/processed/manifest.json")
7 PENDING_CHANGES_PATH = str(BASE_DIR / "data/processed/
    pending_changes.json")
8 RAW_DIR = str(BASE_DIR / "data/raw")
9 UPLOAD_DIR = str(BASE_DIR / "data/uploads")
10 COLLECTION_NAME = "elte_ik"
11 EMBEDDING_MODEL = "all-MiniLM-L6-v2"
12 TOP_K = 5
13 CHUNK_SIZE = 800

```

```

14 CHUNK_OVERLAP          = 150
15 MAX_UPLOAD_BYTES       = 20 * 1024 * 1024    # 20 MB
16 ALLOWED_UPLOAD_TYPES   = {
17     "application/pdf", "text/html",
18     "application/vnd.openxmlformats-officedocument.wordprocessingml
19     .document",
20     "application/msword", "application/octet-stream",
21 }
22
23 ALLOWED_EXTENSIONS      = {".pdf", ".html", ".htm", ".docx"}
24
25 OLLAMA_URL              = os.getenv("OLLAMA_URL", "http://localhost:11434/api/
26     generate")
27
28 OLLAMA_MODEL             = "llama3.2:3b"
29
30 TEMPERATURE             = 0.1
31
32 TIMEOUT_S                = 120

```

Code 4.1: Central configuration (app/settings.py)

Table 4.4 groups the most commonly adjusted parameters by concern.

Parameter	Default	Description
<i>Retrieval and chunking</i>		
TOP_K	5	Chunks retrieved per query.
CHUNK_SIZE	800	Maximum chunk size in characters.
CHUNK_OVERLAP	150	Overlap between adjacent chunks in characters.
EMBEDDING_MODEL	all-MiniLM-L6-v2	Sentence embedding model. Must be identical between build_index.py and the serving layer; a mismatch produces incorrect retrieval.
<i>Language model</i>		
OLLAMA_MODEL	llama3.2:3b	Default language model passed to Ollama.
TEMPERATURE	0.1	LLM sampling temperature.
TIMEOUT_S	120	Per-request timeout for Ollama calls in seconds.
<i>Upload validation</i>		
MAX_UPLOAD_BYTES	20 MB	Maximum accepted file size for user uploads.
ALLOWED_EXTENSIONS	.pdf .html .docx	File extensions accepted by the upload endpoint.

Table 4.4: Key configuration parameters in app/settings.py.

`OLLAMA_URL` is the only parameter read from the environment rather than hardcoded. Its default value points to `localhost`, which is correct for the manual setup path. The Docker Compose file overrides it with `http://ollama:11434/api/generate` so that the application container reaches the Ollama container by service name without any code change. This single environment variable is what makes the same image work in both deployment modes.

4.2.3 Data Collection and Ingestion

ELTE Faculty of Informatics publishes its information across static HTML pages, downloadable PDFs, and DOCX documents. The data pipeline collects and converts all three formats into a uniform text representation before any machine learning component is applied.

File Loaders

Each file type requires a different parsing strategy. PDFs are loaded with PyMuPDF page by page. HTML files are parsed with BeautifulSoup: structural noise tags (`script`, `style`, `nav`, `footer`) are stripped, then each paragraph is prefixed by its nearest heading to preserve section context. DOCX files are parsed with `python-docx`, extracting paragraphs and tables while tracking heading structure. All formats share a cleaning step that collapses blank lines and normalises whitespace.

```
1 def clean_text(text: str) -> str:
2     text = re.sub(r"\n{3,}", "\n\n", text)
3     text = re.sub(r"[\t]+", " ", text)
4     return text.strip()
5
6 def load_file(file_path: str | Path) -> List[Document]:
7     suffix = Path(file_path).suffix.lower()
8     if suffix == ".pdf":
9         return load_pdf_pymupdf(file_path)
10    elif suffix in {".txt", ".md"}:
11        return load_txt(file_path)
12    elif suffix in {".html", ".htm"}:
13        return load_html(file_path)
14    elif suffix == ".docx":
15        return load_docx(file_path)
16    else:
17        raise ValueError(f"Unsupported file type: {suffix}")
```

Code 4.2: Text cleaning and unified file loader (`scripts/build_index.py`)

Every document is represented as a `LangChain Document` carrying its text and a metadata dictionary (file name, file type, page number, and `source_relative` path). This metadata is preserved into ChromaDB and surfaced to the user as source references on every answer.

Web Crawler

Raw documents are collected by `scripts/crawler.py`, which performs a resumable breadth-first crawl across two domains: all English pages under `inf.elte.hu` recursively, and twenty whitelisted path subtrees on `www.elte.hu` covering Neptun registration, visa procedures, student finances, and dormitory housing. Explicit path prefixes on the second domain prevent the crawler from wandering into unrelated university-wide content.

The crawler confirms each page is English via the `lang` attribute and URL path before saving it. Linked PDFs and DOCX files are downloaded separately into `linked_pdfs/` and `linked_docx/`. A 0.5-second delay between requests limits server load. The crawler is resumable — existing files are skipped — and a `-reset` flag starts fresh. The final corpus contains 482 files.

Breadth-First Search Traversal

BFS is implemented with a `deque` as the frontier queue. It explores pages level by level, ensuring shallower and typically more important pages are crawled before deep sub-pages, while bounding memory to the width of the graph rather than its depth.

Algorithm 1 BFS Web Crawler**Require:** Seeds S , allowed-domain predicate $allowed(\cdot)$

```

1:  $visited \leftarrow \emptyset$ 
2:  $queue \leftarrow \text{DEQUEUE}(S)$ 
3: while  $queue$  is not empty do
4:    $u \leftarrow queue.popleft()$ 
5:   if  $u \in visited$  then continue
6:   end if
7:    $visited += \{u\}$ 
8:   if  $u$  already on disk then parse cached HTML; goto link extraction
9:   end if
10:   $html \leftarrow \text{FETCH}(u)$ 
11:  if  $html$  is English then save to data/raw/
12:  end if
13:  for each link  $v$  extracted from  $html$  do
14:    if  $v$  ends in .pdf or .docx and  $allowed(v)$  then
15:      download  $v$  to linked_pdfs/ or linked_docx/
16:    else if  $allowed(v)$  and  $v \notin visited$  then
17:       $queue.append(v)$ 
18:    end if
19:  end for
20: end while

```

Chunking Strategy

A language model has a limited context window, so documents are split into overlapping chunks using LangChain's `RecursiveCharacterTextSplitter`.

```

1 text_splitter = RecursiveCharacterTextSplitter(
2     chunk_size=800,
3     chunk_overlap=150,
4     length_function=len,
5     separators=["\n\n", "\n", ". ", " ", ""]
6 )

```

Code 4.3: Chunking configuration (`scripts/build_index.py`)

800 characters is large enough to contain a complete regulation clause yet small enough for the embedding model to encode with high fidelity. The 150-character overlap ensures that no sentence is silently dropped at a split boundary. The separator list defines a priority order — paragraph breaks first, then line breaks, then sentence ends — falling back to mid-word splits only as a last resort. Chunks shorter than 100 characters are discarded, and each surviving chunk is assigned a `chunk_id` of the form `<source_relative>::chunk_N`.

Incremental Ingestion with SHA-256 Manifest

Re-processing all 482 source files from scratch on every run would be wasteful. `build_index.py` therefore maintains a manifest at `data/processed/manifest.json` recording a content hash, chunk count, and timestamp for every ingested file.

The hash function used is SHA-256, which maps any file's binary content to a fixed 256-bit fingerprint. Any change to even a single byte produces a completely different fingerprint, making it a reliable change detector. SHA-256 was chosen over simpler alternatives such as file size or modification timestamp because it is robust to files that are re-downloaded with the same timestamp but different content.

On each run the script computes the hash of every file on disk and compares it against the manifest, producing three change sets: new, updated, and deleted files. Only new and updated files are re-chunked.

```
1 new_files      = sorted(disk_paths - manifest_paths)
2 deleted_files = sorted(manifest_paths - disk_paths)
3 updated_files = sorted(
4     p for p in (disk_paths & manifest_paths)
5     if manifest[p]["content_hash"] != current_files[p][1]
6 )
```

Code 4.4: Incremental diff logic (`scripts/build_index.py`)

The diff results are written to `pending_changes.json`. In Phase 2 of the same run, the embedding step reads this file to determine exactly which chunk IDs to delete from ChromaDB and which new vectors to insert, avoiding a full re-embed of the collection. Once complete, `pending_changes.json` is deleted so the next run starts clean.

4.2.4 Embedding and Vector Storage

Each text chunk is converted into a numeric vector — an *embedding* — that captures its semantic meaning. Chunks with similar meaning produce similar vectors, allowing the retrieval step to find relevant content even when the user's phrasing differs from the exact wording in the source document.

Embedding Model

The chosen model is `all-MiniLM-L6-v2`, a compact sentence transformer that produces 384-dimensional vectors and runs efficiently on CPU. `build_index.py` encodes all chunks in a single batched call:

```
1 class EmbeddingPipeline:
2     def __init__(self, model_name: str = EMBEDDING_MODEL):
3         self.model = SentenceTransformer(model_name)
4
5     def encode(self, texts: list[str]) -> np.ndarray:
6         return self.model.encode(texts, show_progress_bar=True)
```

Code 4.5: Embedding pipeline (`scripts/build_index.py`)

The same model is loaded as a singleton in the serving layer so that document and query embeddings always live in the same vector space. A mismatch between the model used at index time and at query time produces silently incorrect retrieval results.

ChromaDB Vector Store

The embeddings, texts, and metadata are stored in ChromaDB, a lightweight vector database that persists to disk and requires no separate server process. A single collection named `elte_ik` is created with cosine similarity as the distance metric.

```
1 client = chromadb.PersistentClient(path=CHROMA_PATH)
2 collection = client.get_or_create_collection(
3     name=COLLECTION_NAME,
4     metadata={"hnsw:space": "cosine"}
5 )
6 collection.upsert(
7     ids=[str(c["metadata"]["chunk_id"]) for c in chunks],
8     embeddings=embeddings.tolist(),
9     documents=texts,
10    metadatas=[c["metadata"] for c in chunks]
11 )
```

Code 4.6: Collection creation and upsert (`scripts/build_index.py`)

`upsert` rather than `add` ensures that re-running `build_index.py` after new documents have been crawled updates existing chunks in place rather than creating duplicates.

Cosine Similarity

For a query embedding $\mathbf{q}$ and a chunk embedding $\mathbf{d}$, cosine similarity is defined as:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \quad (4.1)$$

The result lies in $[-1, 1]$, where 1 means maximally similar and 0 means unrelated. It is preferred over Euclidean distance for sentence embeddings because it is invariant to vector magnitude — two chunks expressing the same concept in different amounts of text score similarly, whereas Euclidean distance would penalise the longer chunk for having a larger-norm embedding.

HNSW Index

ChromaDB indexes the stored vectors using the Hierarchical Navigable Small World (HNSW) graph [4], which performs approximate nearest-neighbour search in sub-linear time. HNSW builds a multi-layer proximity graph and navigates it greedily from a random entry point, converging on the nearest neighbours without examining every element. At 6128 stored chunks the brute-force alternative would also be fast enough, but HNSW ensures the system remains performant as the collection grows.

4.2.5 Retrieval-Augmented Generation Pipeline

The RAG pipeline is the core intelligence of the system. It consists of three steps that run on every incoming user query: retrieval, prompt construction, and LLM generation. All three are implemented in `app/rag.py`.

Retrieval

The ChromaDB client, collection, and embedding model are initialised as module-level singletons at import time, so they are loaded from disk once when the FastAPI process starts and reused across all requests. Both `SentenceTransformer`

and ChromaDB's `PersistentClient` are safe for concurrent read access, so no locking is required under uvicorn's default threaded worker.

```
1 _client      = chromadb.PersistentClient(path=CHROMA_PATH)
2 _collection = _client.get_collection(name=COLLECTION_NAME)
3 _model      = SentenceTransformer(EMBEDDING_MODEL)
4
5 def retrieve(query: str, n_results: int = TOP_K) -> list[dict]:
6     query_emb = _model.encode([query]).tolist()
7     results   = _collection.query(
8         query_embeddings=query_emb, n_results=n_results
9     )
10    return [
11        {"content": doc, "metadata": meta}
12        for doc, meta in zip(
13            results["documents"][0], results["metadatas"][0]
14        )
15    ]
```

Code 4.7: Singletons and retrieval function (app/rag.py)

When a query arrives, the embedding model encodes it into a 384-dimensional vector and ChromaDB returns the `TOP_K` (default: 5) chunks with the highest cosine similarity to that vector.

Prompt Construction

The retrieved chunks are assembled into a structured prompt that instructs the LLM to answer strictly from the provided context and to acknowledge when the answer is absent. Each chunk is labelled with its source file name.

```
1 def build_prompt(query: str, chunks: list[dict]) -> str:
2     context = "\n\n".join(
3         f"[Source: {c['metadata']['file_name']}] \n {c['content']}"
4         for c in chunks
5     )
6     return (
7         "You are a helpful assistant for ELTE Faculty of\n\n"
8         "Informatics "\n\n"
9         "students. Answer the question using only the context below\n\n"
10        ". "\n\n"
11        "If the answer is not in the context, say so.\n\n"
```

```
10     f"Context:\n{context}\n\n"  
11     f"Question: {query}\nAnswer:"  
12 )
```

Code 4.8: Prompt template (app/rag.py)

The instruction “*If the answer is not in the context, say so*” is critical for a faculty assistant: acknowledging a gap in the indexed documents is always preferable to hallucinating a regulation or deadline.

LLM Generation via Ollama

Answers are generated by a locally running model served through Ollama’s HTTP API. The default model is `llama3.2:3b`, but the system supports runtime model switching — the selected model name is passed with each request and forwarded to `call_ollama`.

```
1 def call_ollama(prompt: str, model: str = OLLAMA_MODEL) -> str:  
2     if not check_ollama():  
3         raise RuntimeError("Ollama is not running.")  
4     payload = {  
5         "model": model, "prompt": prompt, "stream": False,  
6         "options": {"temperature": TEMPERATURE, "num_ctx": 2048},  
7     }  
8     try:  
9         r = requests.post(OLLAMA_URL, json=payload, timeout=  
10             TIMEOUT_S)  
11         r.raise_for_status()  
12         return r.json()["response"].strip()  
13     except requests.exceptions.Timeout:  
14         raise RuntimeError(f"Ollama timed out after {TIMEOUT_S}s")  
15     except requests.exceptions.HTTPError as e:  
16         raise RuntimeError(  
17             f"Ollama HTTP {e.response.status_code} -- "  
18             f"is model '{model}' pulled?"  
19         )  
20     except KeyError:  
21         raise RuntimeError(f"Unexpected Ollama response: {r.text[:200]}")
```

Code 4.9: Ollama invocation (app/rag.py)

The context window is fixed at 2048 tokens, which fits five retrieved chunks plus the prompt template with room for the generated answer. The `try/except` block distinguishes three failure modes — timeout (model still loading), HTTP error (model not pulled), and malformed response body — each producing a descriptive `RuntimeError` that the API layer converts into an HTTP 503.

End-to-End Query Function

The three steps are composed into `rag_query`, the single entry point called by the API layer:

```
1 def rag_query(query: str, model: str = OLLAMA_MODEL) -> dict:
2     chunks = retrieve(query)
3     prompt = build_prompt(query, chunks)
4     answer = call_ollama(prompt, model=model)
5     return {
6         "answer": answer,
7         "sources": [
8             {"chunk_id": c["metadata"]["chunk_id"],
9              "file":      c["metadata"]["file_name"]}
10            for c in chunks
11        ],
12    }
```

Code 4.10: End-to-end RAG query (`app/rag.py`)

The function returns both the answer and a list of source references, allowing the frontend to display which documents were used and giving the user a way to verify or follow up directly.

4.2.6 FastAPI Backend

The backend is a FastAPI application (`app/main.py`) that wires all HTTP endpoints to the `rag`, `ingest`, and `logger` modules and serves the static frontend. It contains no business logic of its own.

Method	Path	Description
GET	/health	Returns application status and whether Ollama is reachable.
GET	/models	Returns the list of models available in the local Ollama installation.
GET	/info	Returns the active model, embedding model, total chunk count, and list of user-uploaded documents.
POST	/chat	Accepts a message, optional <code>session_id</code> and <code>model</code> , runs the RAG pipeline, logs the interaction, and returns the answer with source references.
GET	/sessions	Returns all past sessions ordered by most recently updated.
GET	/sessions/{id}/messages	Returns all messages in a session in chronological order.
DELETE	/sessions/{id}	Deletes a session and all its associated messages.
POST	/upload	Validates, chunks, embeds, and indexes a user-uploaded file into the live ChromaDB collection.
GET	/source/{filename}	Serves the original source file from <code>data/raw/</code> or <code>data/uploads/</code> for direct inspection.

Table 4.5: API endpoints exposed by the FastAPI backend.

Chat Endpoint

The `/chat` endpoint is the primary entry point for all user queries. It delegates to `rag_query`, logs the result, and updates the session record.

```

1 @app.post("/chat", response_model=ChatResponse)
2 def chat(req: ChatRequest):
3     start = time.time()
4     try:
5         result = rag_query(req.message, model=req.model or
6                             OLLAMA_MODEL)
7     except RuntimeError as e:
8         raise HTTPException(status_code=503, detail=str(e))
9
10    response_ms = int((time.time() - start) * 1000)
11
12    if req.session_id:

```



```
12     log_chat(  
13         user_message=req.message,  
14         answer=result["answer"],  
15         sources=result["sources"],  
16         response_ms=response_ms,  
17         session_id=req.session_id,  
18     )  
19     upsert_session(req.session_id, req.message)  
20  
21     return {**result, "response_ms": response_ms}
```

Code 4.11: Chat endpoint (app/main.py)

The model name is forwarded from the request, falling back to `OLLAMA_MODEL` if absent — this is how the frontend’s model selector takes effect without requiring a server restart. If `rag_query` raises a `RuntimeError` (Ollama not running, timed out, or model not pulled), the endpoint converts it into an HTTP 503. Logging and session update only occur when a `session_id` is provided; requests from scripts or other API clients that omit it are answered without being persisted.

Upload Endpoint

The `/upload` endpoint extends the knowledge base at runtime without requiring a pipeline re-run or server restart.

```
1 @app.post("/upload", response_model=UploadResponse)  
2 def upload_file(file: UploadFile = File(...)):  
3     ext = Path(file.filename).suffix.lower()  
4     if file.content_type not in ALLOWED_UPLOAD_TYPES \  
5         or ext not in ALLOWED_EXTENSIONS:  
6         raise HTTPException(status_code=415, detail="Unsupported  
7             file type.")  
8  
9     content = file.file.read(MAX_UPLOAD_BYTES + 1)  
10    if len(content) > MAX_UPLOAD_BYTES:  
11        raise HTTPException(status_code=413, detail="File too large  
12            .")  
13  
14    sha256 = hashlib.sha256(content).hexdigest()  
15    if ingest.is_already_indexed(sha256):  
16        return UploadResponse(status="already_indexed", ...)
```

```
15
16     dest = Path(UPLOAD_DIR) / f"{sha256[:12]}_{safe_name}"
17     dest.write_bytes(content)
18     report = ingest.ingest_file(dest)
19     log_upload(file_name=safe_name, sha256=sha256,
20               size_bytes=len(content), chunk_count=report.
21                 chunk_count,
22                 status="indexed")
23     return UploadResponse(status="indexed", ...)
```

Code 4.12: Upload endpoint (`app/main.py`)

The endpoint processes each upload in five stages:

1. **Type validation** — the declared content type and file extension are checked against the allowed lists in `settings.py`; only PDF, HTML, and DOCX are accepted.
2. **Size check** — the file is read into memory and rejected if it exceeds `MAX_UPLOAD_BYTES` (20 MB).
3. **Deduplication** — a SHA-256 digest is computed and checked against the manifest; if the file is already indexed the endpoint returns immediately without re-processing.
4. **Ingestion** — the file is written to `data/uploads/` and passed to `ingest.ingest_file()`, which applies the same loaders, chunking, and embedding logic as the data pipeline and upserts the vectors directly into the running ChromaDB collection.
5. **Audit log** — the upload is recorded in the `uploads` table with its digest, size, and chunk count.

The document is immediately available for retrieval with no server restart required.

CORS middleware is enabled globally so that the frontend, served as a static file by the same FastAPI process, can communicate with the API without browser origin restrictions.

4.2.7 Logging System

Every interaction is recorded in a SQLite database at `data/logs/chat_logs.db`. The schema consists of three tables.

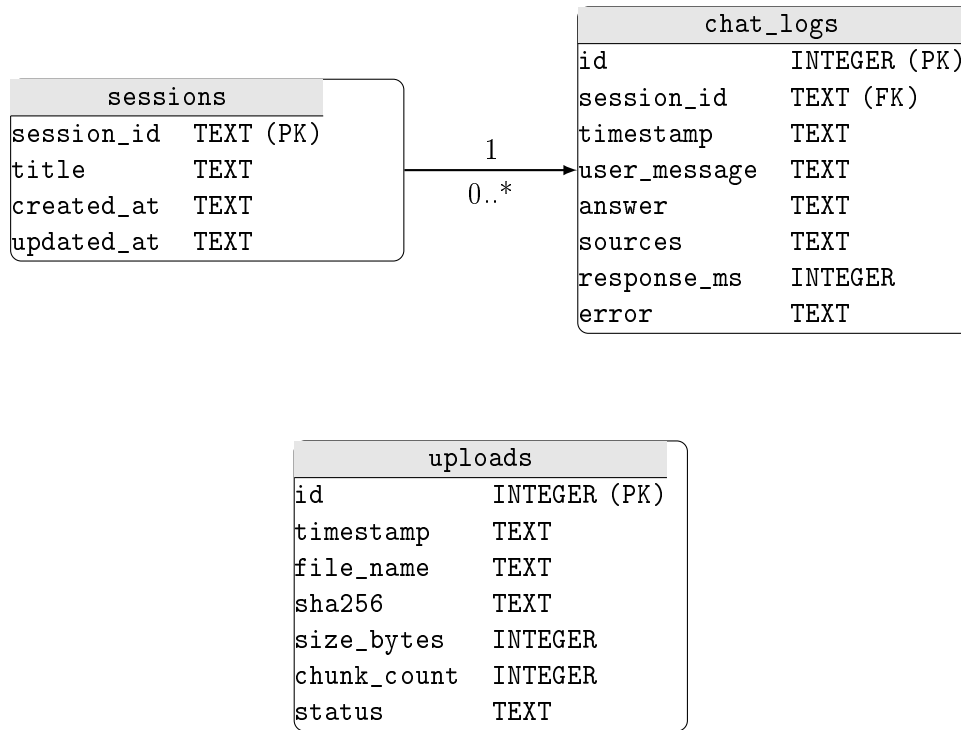


Figure 4.3: Entity-relationship diagram for the SQLite logging database.

The `chat_logs` table links each message to a `session_id`, enabling per-session history retrieval. The `sessions` table stores one row per conversation with a human-readable title derived from the opening message, so the sidebar can list sessions without scanning the full message log. The `uploads` table provides an audit trail for the live document ingestion feature, recording each file’s digest and chunk count alongside its ingestion status.

4.2.8 Frontend

The user interface is a single-page HTML application served as a static file by FastAPI. Vanilla JavaScript with no framework dependency eliminates a build step and all Node.js tooling — deployment is a single file copy.

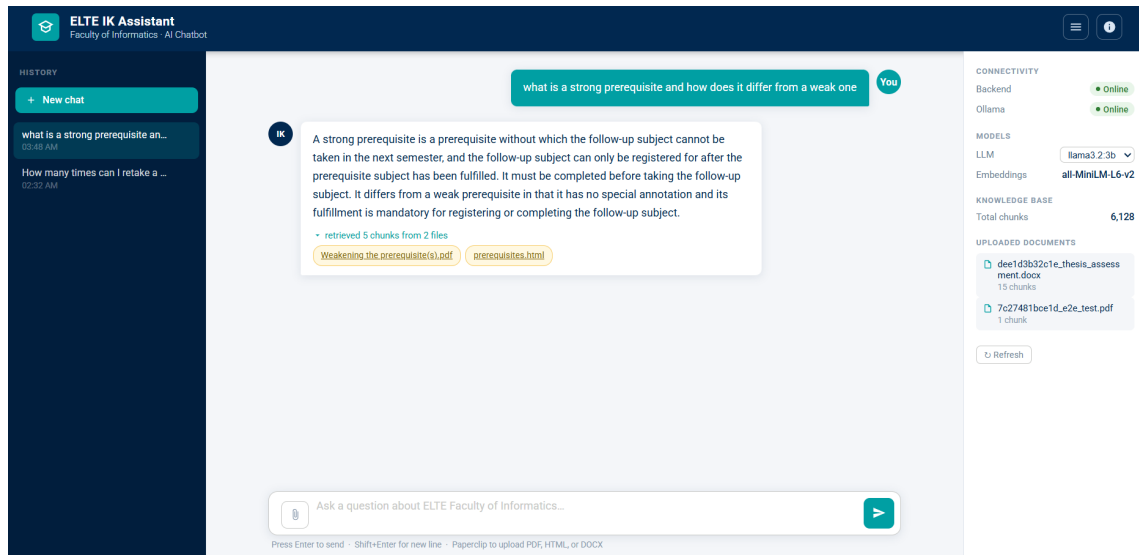


Figure 4.4: The ELTE IK Assistant web interface.

Layout and visual design

A fixed header bar, scrollable message area, and pinned input row at the bottom. The header follows the ELTE colour palette — navy blue (#012850) for structure, teal (#009FA3) for interactive elements — and shows an Ollama status indicator that polls `/health` on page load.

Session management

A UUID is generated with `crypto.randomUUID()` on page load and sent with every `/chat` request. Past sessions are listed in the left sidebar, loaded from `GET /sessions`; clicking one replays its history from `GET /sessions/{id}/messages`. New `chat` generates a fresh UUID and clears the display.

Message flow

On submit, a typing indicator is appended while `POST /chat` is in flight. On success it is replaced by the answer bubble, which includes a collapsible source list — each filename links to `GET /source/{filename}` so the user can open the original document. On HTTP 503 an inline error replaces the indicator.

Model selection

`GET /models` populates a dropdown on page load. The selected model is stored in a module-level variable and included in every `/chat` request. Switching takes effect

immediately.

Document upload

A hidden `<input type="file">` filtered to PDF, HTML, and DOCX posts to `POST /upload` as `multipart/form-data`. The response `chunk_count` is shown as confirmation, or `already_indexed` if the file is a duplicate.

Chapter 5

Testing and Evaluation

This chapter covers testing and evaluation of the ELTE IK Assistant. It begins with functional testing of the API endpoints and pipeline, then presents an empirical system evaluation across four model and retrieval configurations.

5.1 Functional Testing

The project includes an end-to-end test suite located in the `tests/` directory. Tests are written using `pytest` and interact with the live system over HTTP — there are no mocks — so they exercise the full request path from the API layer through ChromaDB retrieval to Ollama inference.

Test Architecture

All tests are tagged with the `@pytest.mark.e2e` marker, which keeps them separate from any future unit tests and allows selective execution. A session-scoped `warm_up_model` fixture fires one dummy `/chat` request before the first test runs, loading the LLM into RAM so that individual tests do not time out during model initialisation. Two further session-scoped fixtures — `live_server` and `ollama_up` — check the `/health` endpoint and skip the entire session with an actionable message if the server or Ollama is not reachable.

Coverage

Table 5.1 summarises the endpoints and behaviours verified by the suite.

Endpoint	Test	Assertion	Result
GET /health	server up	<code>status == "ok"</code> , <code>ollama</code> key present	Pass
GET /health	Ollama online	<code>ollama == true</code>	Pass
POST /chat	basic response	200, non-empty answer, sources list, <code>response_ms > 0</code>	Pass
POST /chat	in-scope query	answer length > 20 characters	Pass
POST /chat	out-of-scope query	200, non-empty answer	Pass
POST /chat	source keys	each source contains <code>chunk_id</code> and <code>file</code>	Pass
POST /chat	session logging	session appears in GET /sessions after chat	Pass
GET /sessions	session created	session ID present after chat with <code>session_id</code>	Pass
GET /sessions/{id}/messages	messages retrievable	one message with correct <code>user_message</code> field	Pass
DELETE /sessions/{id}	session delete	204, session absent from list afterwards	Pass
POST /upload	valid PDF	200, <code>status</code> is "indexed" or "already_indexed"	Pass
POST /upload	duplicate file	<code>status == "already_indexed"</code> on second upload	Pass
POST /upload	invalid extension	415 Unsupported Media Type	Pass
GET /info	collection size	<code>total_chunks</code> is a non-negative integer	Pass
GET /info	embedding model	<code>embedding_model == "all-MiniLM-L6-v2"</code>	Pass
GET /models	model list	<code>models</code> is a non-empty list, <code>default</code> key present	Pass

Table 5.1: End-to-end test cases and their assertions.

Because LLM output is non-deterministic, no test asserts exact answer text; all assertions target response structure, status codes, and field types. All 16 test cases passed on the reference hardware configuration running `llama3.2:3b` via Ollama.

Running the Tests

Before running the suite the following prerequisites must be met: Ollama must be running (`ollama serve`), the target model must be pulled (`ollama pull llama3.2:3b`), and the FastAPI server must be started (`uvicorn app.main:app --reload`). The suite is then executed with:

```
1 python -m pytest tests
```

Code 5.1: Upload endpoint (`app/main.py`)

If the server or Ollama is not available, all tests are automatically skipped rather than failed, and the skip message states the exact command needed to resolve the issue.

5.2 System Evaluation

The following sections present an empirical evaluation of the system across four configurations, measuring answer quality, out-of-scope refusal behaviour, retrieval effectiveness, and response latency. The goal is to quantify how much value the RAG component adds over a plain language model baseline, and to compare the two evaluated local models under both conditions.

5.2.1 Evaluation Setup

Configurations

Four configurations were evaluated in a factorial design crossing two models with two retrieval modes:

ID	Model	RAG	Description
A1	llama3.2:3b	No	Baseline: model knowledge only
B1	llama3.2:3b	Yes	RAG with top-5 chunk retrieval
A2	gemma3:4b	No	Baseline: model knowledge only
B2	gemma3:4b	Yes	RAG with top-5 chunk retrieval

Table 5.2: The four evaluation configurations.

Both models were run locally via Ollama with 4-bit quantisation and temperature 0.1. No internet access was used during evaluation.

Test Set

The test set consists of 20 questions divided into two categories.

15 in-scope questions covering topics present in the crawled ELTE documents: course prerequisite rules, semester status definitions, financial support, student card eligibility, housing, emergency contacts, the Stipendium Hungaricum scholarship,

exam registration rules, BSc credit requirements, Erasmus eligibility, and health insurance. Each question has a reference answer written manually by consulting the source documents directly.

5 out-of-scope questions with no connection to the faculty corpus: current weather, a sports result, cooking instructions, national population statistics, and a movie recommendation. These measure whether the system correctly declines to answer rather than generating an unsupported response.

Each question was run through all four configurations independently with no shared conversation history between questions. The full list of questions, categories, and reference answers is provided in Appendix A.

Metrics

ROUGE-L The longest-common-subsequence F1 score between the generated answer and the reference answer [5]. Captures lexical overlap with the ground truth on a scale from 0 to 1. Computed on in-scope questions only.

Semantic similarity Cosine similarity between the embedding of the reference answer and the embedding of the generated answer, computed using the same `all-MiniLM-L6-v2` model used for retrieval. Addresses ROUGE-L’s main weakness: a correct answer phrased differently still scores well. Computed on in-scope questions only.

Refusal rate The fraction of out-of-scope questions for which the model produced a non-answer. Detected automatically using a phrase list; higher is better for this category.

Retrieval hit-rate For RAG configurations: the fraction of in-scope questions for which at least one retrieved chunk came from the expected source document. Measures retrieval quality independently of the language model. Because retrieval depends only on the embedding model and vector index, this value is identical across all four configurations.

Response latency Wall-clock time from request to complete response, measured in seconds on the reference hardware with no GPU.

Hardware

All runs were performed on the development machine whose specifications are listed in Table 4.2. No GPU acceleration was used; all inference ran on CPU.

5.2.2 Results

Overall Metric Summary

Table 5.3 reports the per-configuration averages across all 20 test questions.

ID	Model	ROUGE-L	Sem Sim	Refusal	Hit-rate	Avg time
A1	llama3.2:3b, no RAG	0.128	0.631	0.0	0.933	41.5 s
B1	llama3.2:3b, RAG	0.319	0.757	0.8	0.933	53.2 s
A2	gemma3:4b, no RAG	0.076	0.618	0.0	0.933	157.2 s
B2	gemma3:4b, RAG	0.357	0.778	1.0	0.933	66.9 s

Table 5.3: Evaluation results averaged over 20 test questions. Hit-rate is computed over in-scope questions only (15 questions); refusal rate is computed over out-of-scope questions only (5 questions).

Effect of RAG on Answer Quality

Adding RAG context produces a consistent and substantial improvement in both lexical and semantic quality for both models.

ROUGE-L increases by a factor of $2.5\times$ for llama3.2:3b ($0.128 \rightarrow 0.319$) and by $4.7\times$ for gemma3:4b ($0.076 \rightarrow 0.357$). The larger relative gain for Gemma reflects its greater reliance on parametric knowledge without retrieval: without grounding, it produces generic answers that share little lexical overlap with the ELTE-specific reference answers.

Semantic similarity shows the same direction with a smaller relative gap, which is expected — both the no-RAG answers and the references discuss the same topic, so embedding-level similarity remains moderate even without retrieval. RAG still moves both models upward: $0.631 \rightarrow 0.757$ for Llama and $0.618 \rightarrow 0.778$ for Gemma.

Model Comparison

Among the RAG configurations, gemma3:4b (B2) outperforms llama3.2:3b (B1) on both answer quality metrics: ROUGE-L 0.357 vs. 0.319 and semantic similarity

0.778 vs. 0.757. Gemma produces more concise answers that stay closer to the retrieved context, whereas Llama tends to expand its answers with additional commentary that reduces lexical overlap with the reference.

Among the no-RAG baselines, both models score poorly and comparably. Llama has a slight advantage on ROUGE-L (0.128 vs. 0.076) but the difference is negligible in practice: neither baseline reliably answers ELTE-specific questions without retrieval.

Retrieval Quality

The retrieval hit-rate is 0.933 (14 out of 15 in-scope questions) and is identical across all four configurations, since retrieval depends solely on the embedding model and vector index. The single missed question is Q14 (*Can a Stipendium Hungaricum student go on Erasmus Scholarship?*), where the retrieved chunks did not include the expected `info-for-outgoing-erasmus-students` page. The relevant content exists in the corpus but the embedding-based search ranked other Erasmus-related chunks higher for this query.

Out-of-Scope Refusal

Neither baseline configuration refuses any out-of-scope question: A1 and A2 answer all five irrelevant queries confidently, generating plausible but entirely off-topic responses. Among the RAG configurations, refusal rates are substantially higher. `llama3.2:3b` with RAG (B1) correctly refuses 4 out of 5 out-of-scope questions (80%). The single slip is Q19 (*What is the population of Hungary?*), where a population figure appeared incidentally in a retrieved chunk about Hungarian student demographics, giving the model enough apparent grounding to produce an answer. `gemma3:4b` with RAG (B2) refuses all 5 out-of-scope questions (100%).

In both RAG configurations the refusal mechanism is the same: when no relevant chunks are retrieved, the model follows the prompt instruction to answer only from provided context and declines rather than falling back on parametric knowledge.

Response Latency

Configuration	Mean (s)	Median (s)	Min (s)	Max (s)
A1 — llama3.2:3b, no RAG	41.5	42.5	13.1	86.4
B1 — llama3.2:3b, RAG	53.2	51.7	24.0	88.5
A2 — gemma3:4b, no RAG	157.2	158.3	10.7	242.5
B2 — gemma3:4b, RAG	66.9	66.3	44.2	102.2

Table 5.4: Response latency in seconds. The non-functional requirement NF-4 sets a target of 30 seconds.

All four configurations exceed the 30-second latency target defined in NF-4. Two observations stand out. First, RAG adds latency for `llama3.2:3b` (+11.7 s) but *reduces* it substantially for `gemma3:4b` (−90.3 s): when grounded by retrieved context Gemma produces shorter, more focused answers, whereas without context it generates lengthy elaborations. Second, `gemma3:4b` without RAG is effectively unusable on this hardware at a mean of 157 seconds and a maximum of over four minutes, while with RAG it falls to 67 seconds — still above the target but within a practical range.

5.2.3 Discussion

RAG Effectiveness

The results confirm that RAG is effective for this use case. Grounding the language model in retrieved faculty documents more than doubles ROUGE-L scores for both models and raises semantic similarity by roughly 0.14–0.16 points. The improvements are consistent across both models and large enough to be meaningful in practice, not merely statistical noise over a 20-question benchmark.

The mechanism is straightforward. Without context, neither `llama3.2:3b` nor `gemma3:4b` has reliable knowledge of ELTE-specific regulations — they were trained on general web text and have no exposure to the faculty’s prerequisite rules, credit requirements, or administrative procedures. The retrieval step injects the relevant passage directly into the prompt, allowing the model to produce a grounded answer even though it has no parametric knowledge of the topic.

RAG also has a clear secondary benefit: it substantially improves the system’s ability to decline out-of-scope questions. The no-RAG baselines offer no protection at all, while both RAG configurations refuse the majority of irrelevant queries. This is a

direct consequence of the prompt instruction to answer only from provided context: when no relevant chunks are retrieved, the model has no grounding material to draw on and declines rather than hallucinating.

Recommended Configuration

Based on the evaluation results, **gemma3:4b with RAG (B2)** is the recommended configuration: it achieves the highest answer quality (ROUGE-L 0.357, semantic similarity 0.778), a perfect out-of-scope refusal rate, and — counterintuitively — lower latency than its own no-RAG baseline (67 s vs. 157 s). If latency is the primary constraint, **llama3.2:3b with RAG (B1)** offers a reasonable trade-off at 53 seconds mean response time with a modest quality reduction and a refusal rate of 0.8.

Limitations

Several limitations of this evaluation should be acknowledged.

Small benchmark. Twenty questions is a limited evaluation set. The in-scope questions cover eight source documents but are skewed toward topics with well-structured source material. Topics such as exam appeal procedures, credit transfer rules, and dormitory application deadlines are not represented.

ROUGE-L as a proxy. ROUGE-L measures lexical overlap with a manually written reference answer. A response that is semantically correct but uses different phrasing will score poorly. Semantic similarity partially addresses this, but both metrics share a deeper limitation: the reference answers were written by consulting the same source documents that the retrieval index is built from, so high scores partly reflect how closely the model copies retrieved text rather than how useful the answer would be to a real student.

Heuristic refusal detection. Refusal rate is computed automatically using a phrase list. This approach can both under-count refusals (if the model uses an unanticipated phrasing) and over-count them (if a phrase appears in an answer that is not actually a refusal). The Q19 case — where llama3.2:3b with RAG answered a population question because a relevant figure appeared incidentally in a retrieved

chunk — illustrates a related edge case: the model was not hallucinating, but the answer was not grounded in faculty-specific content.

Latency. All configurations exceed the 30-second NF-4 target. On CPU-only hardware the bottleneck is autoregressive token generation, which scales with output length. A deployment on shared faculty hardware with a GPU would reduce latency significantly; alternatively, a smaller 1B-parameter model could be evaluated as a faster option at the cost of some answer quality.

Chapter 6

Conclusion

This thesis has presented the design, implementation, and empirical evaluation of a locally deployed retrieval-augmented generation chatbot for the ELTE Faculty of Informatics, enabling students and staff to ask natural-language questions about curriculum, prerequisites, exam rules, and Neptun procedures without sending any data to an external server.

6.1 Summary of Contributions

Document collection and ingestion pipeline. A resumable breadth-first crawler and an incremental ingestion pipeline with SHA-256 change detection built a 6112-chunk ChromaDB index from 482 faculty documents, ensuring that subsequent runs only reprocess new or modified files.

Offline RAG pipeline. Query responses are grounded in the index using all-MiniLM-L6-v2 embeddings and top-5 cosine retrieval. The entire pipeline runs on a standard laptop without GPU acceleration or any external network dependency at inference time.

Local language model serving. Two quantised local models — llama3.2:3b and gemma3:4b — are served via Ollama at 4-bit quantisation. No query, retrieved chunk, or generated answer leaves the host machine during inference.

Production-quality backend and frontend. A FastAPI backend exposes the pipeline through a REST API, logs interactions to a local SQLite database, and

serves a browser-based chat interface matching the faculty’s visual identity. A 16-case end-to-end test suite verified the full request path from API layer to Ollama inference; all tests passed.

Empirical evaluation. A 20-question benchmark across four configurations confirmed that RAG is the decisive factor in answer quality. ROUGE-L improves by $2.5\times$ for `llama3.2:3b` and $4.7\times$ for `gemma3:4b` when retrieval context is added. The best configuration, `gemma3:4b` with RAG, achieves ROUGE-L 0.357, semantic similarity 0.778, and a perfect out-of-scope refusal rate of 100%.

6.2 Reflection on Goals

The system meets its two primary goals. All computation runs on the local machine with no external network dependency after the initial model download, satisfying the offline-operation requirement. With RAG, both models produce answers substantially more faithful to faculty source documents than their no-RAG baselines, which have no parametric knowledge of ELTE-specific rules. The retrieval step achieves a hit-rate of 0.933, confirming that the vector index reliably surfaces relevant content.

The 30-second latency target (NF-4) was not met on CPU-only hardware. Mean response times range from 41.5 to 66.9 seconds across the two RAG configurations, and `gemma3:4b` without RAG reaches 157 seconds — effectively unusable. This is a hardware constraint rather than an architectural failure, and is addressed in the future work below.

6.3 Limitations

Retrieval ceiling. The hit-rate of 0.933 leaves one in fifteen questions without the most relevant chunk. The single miss — Q14, on Stipendium Hungaricum and Erasmus eligibility — suggests the chunking strategy does not represent shallow navigation text well. A cross-encoder re-ranking step is the most direct remedy.

Out-of-scope refusal reliability. The prompt-level refusal mechanism has a known gap: `llama3.2:3b` with RAG answered Q19 because a population figure

appeared incidentally in a retrieved chunk, providing apparent grounding for an off-topic response. A retrieval-score threshold below which the system returns a fixed refusal would close this gap more robustly than prompt engineering alone.

Benchmark scale. The 20-question test set is skewed toward prerequisite rules and credit requirements. Topics such as exam appeals, credit transfer, and dormitory applications are not represented, limiting the generalisability of the evaluation results.

6.4 Future Work

University-wide deployment. The system is architecturally ready for deployment beyond a single laptop. On shared faculty hardware with a mid-range GPU, a larger 7B-parameter model would bring latency well within the 30-second target while improving answer quality. Deployment at the university level would also raise the question of corpus governance: the current SQLite-backed document store and ChromaDB index would need to be replaced or augmented with a managed database that supports access control, scheduled re-crawls, and version-tracked document updates. A lightweight administration interface for adding new sources, retiring outdated pages, and triggering incremental re-indexing would be a prerequisite for sustainable operation at that scale.

Retrieval-score threshold for refusal. Comparing the cosine similarity of the top retrieved chunk against a calibrated threshold would provide a robust, model-independent refusal mechanism, eliminating the inconsistency observed with llama3.2:3b and the incidental-match edge case that caused it to answer Q19.

Improved retrieval. A cross-encoder re-ranker or hybrid BM25 + dense retrieval would improve precision on exact-match queries such as course codes and regulation numbers that embedding models handle less reliably than semantic queries.

Multilingual support. Replacing all-MiniLM-L6-v2 with a multilingual sentence encoder would extend the system to Hungarian-language queries without re-indexing the corpus.

User feedback loop. The SQLite interaction log provides a natural foundation for a feedback mechanism. Aggregated helpfulness ratings could identify retrieval failures, surface low-quality chunks, and prioritise document updates.

Appendix A

Evaluation Test Set

Table A.1 lists all 20 questions used in the system evaluation (Section 5.2), together with their category, expected source document, and reference answer. Reference answers for in-scope questions were written manually by consulting the source documents directly. Out-of-scope questions have no reference answer.

Table A.1: Full evaluation test set.

ID	Question	Scope	Source	Reference answer
1	What is a strong prerequisite?	In-scope	prerequisites	Strong prerequisites are prerequisites without which the follow-up subject cannot be taken in the next semester, and the follow-up subject can only be registered for after the prerequisite subject has been fulfilled. Neptun will automatically deregister you from the subject if you do not meet this prerequisite.

ID	Question	Scope	Source	Reference answer
2	What is a weak prerequisite?	In-scope	prerequisites	A weak prerequisite can be taken in the same semester as the follow-up subject, but you must complete it first before you pass the follow-up. If not completed, your grade in the follow-up subject will automatically be failed or unfulfilled even if you pass its exam.
3	If I fail a weak prerequisite, does it count toward my subject registration limit?	In-scope	prerequisites	If you register for the follow-up subject but cannot pass it in a given semester due to a failed prerequisite, the follow-up subject will not be counted into the maximum 3 subject registrations per subject limit during your studies.
4	What is a passive semester at ELTE?	In-scope	Academic Regulations	A passive semester is the semester in which the student announces that he/she wishes to interrupt his/her studies, or cancels active registration, or fails to register until the given deadline. Student status is terminated if the student fails to register for the next semester following two consecutive semesters of suspension.
5	What does active semester mean at ELTE?	In-scope	Academic Regulations	An active semester is the semester in which the student registers to start or continue his/her studies and does not cancel it over the course of the semester.

ID	Question	Scope	Source	Reference answer
6	What financial support is available for ELTE students?	In-scope	Academic Regulations	Financial support takes the following forms: academic scholarship, National Higher Education Scholarship, scientific scholarship, scholarships for competitions and conferences, public service scholarship, sports scholarship, cultural scholarship, compensation for parallel studies, ERASMUS+ Start, “Good student, good athlete” scholarship, professional scholarship, arts scholarship, regular social grant, one-time social grant, Bursa Hungarica, normative grants for foreign students, basic financial support, financial support for traineeships, and doctorate bursary.
7	What is the result of the final examination at ELTE?	In-scope	Academic Regulations	The result of the final examination is the average of the grades given for the elements of the final examination. Any deferring regulation in connection with the grading of the final examination is approved by the Faculty Council on the proposal of the person responsible for the degree course.

ID	Question	Scope	Source	Reference answer
8	What is the student card and how do I get one?	In-scope	Student Card	The student card is a plastic orange-brown card similar in size to a bank card. You are eligible if you are staying at ELTE longer than 12 months, your semester status is active, and your Education ID has appeared in Neptun. To apply, obtain a NEK document from a Kormanyablak office, upload it in Neptun, and register your application electronically in the Neptun system.
9	What housing options are available for ELTE students?	In-scope	housing	The ELTE Dormitory Centre provides accommodation to hundreds of ELTE international students in two cities, Budapest and Szombathely. The dormitories offer high-quality and affordable places and services, with more than half completely rebuilt within recent years, and a vivid community life.
10	What should I do in case of a medical emergency in Hungary?	In-scope	health-insurance	Important emergency numbers: general emergency 112, ambulance 104, fire 105, police 107.

ID	Question	Scope	Source	Reference answer
11	What is the Stipendium Hungaricum scholarship?	In-scope	stipendium	Stipendium Hungaricum is a scholarship programme for foreign students, founded by the Hungarian Government in 2013. The programme aims to promote cultural understanding and economic and cultural relations between Hungary and other countries. The Department of Erasmus+ and International Programmes is responsible for the implementation of the programme at ELTE.
12	What are the rules for exam registration in Neptun?	In-scope	Exam period	You must first register for the subject in Neptun to be eligible for the exam. A maximum of 3 exam attempts per subject are allowed in the same exam period. You can register for an exam and cancel or change exam dates at least 24 hours before the exam starts. Absence without a valid reason is counted as an unsuccessful attempt.
13	How many credits are needed to complete a BSc at ELTE?	In-scope	curriculum	In the BSc programme you have to complete 180 credits: compulsory subjects 127 credits, compulsory elective subjects 23 credits, elective subjects 10 credits, and thesis consultation 20 credits.

ID	Question	Scope	Source	Reference answer
14	Can a Stipendium Hungaricum student go on Erasmus Scholarship?	In-scope	info-for-outgoing-erasmus-students	Stipendium Hungaricum students on BSc and MSc level are not eligible for Erasmus scholarship.
15	What health insurance are ELTE students entitled to?	In-scope	health-insurance	Students should arrive with valid health insurance. EEA students are advised to obtain a European Health Insurance Card before departure. Stipendium Hungaricum, Hungarian Diaspora Scholarship, and Scholarship Programme for Christian Young People holders are eligible for both a TAJ card (non-private) and supplementary private health insurance.
16	What is the weather like in Budapest?	Out-of-scope	—	—
17	Who won the FIFA World Cup in 2022?	Out-of-scope	—	—
18	How do I cook pasta?	Out-of-scope	—	—
19	What is the population of Hungary?	Out-of-scope	—	—
20	Can you recommend a movie to watch?	Out-of-scope	—	—

Bibliography

- [1] University of Michigan. *Generative AI at U-M*. <https://genai.umich.edu>. Accessed: 2026-04-30. 2025.
- [2] University of California, Irvine. *ZotGPT: UCI AI Services Platform*. <https://zotgpt.uci.edu>. Accessed: 2026-04-30. 2025.
- [3] Arizona State University. *CreateAI Platform*. <https://ai.asu.edu/create-ai-platform-product-release-notes>. Accessed: 2026-04-30. 2025.
- [4] Yury A. Malkov and Dmitry A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.
- [5] Chin-Yew Lin. “ROUGE: A Package for Automatic Evaluation of Summaries”. In: *Text Summarization Branches Out*. Association for Computational Linguistics, 2004, pp. 74–81.

List of Figures

2.1	Use case diagram for the ELTE IK Assistant	8
3.1	The ELTE IK Assistant web interface.	16
3.2	The ELTE IK Assistant command-line client.	17
4.1	High-level architecture of the ELTE IK Assistant.	21
4.2	Module structure and dependencies of the ELTE IK Assistant.	23
4.3	Entity-relationship diagram for the SQLite logging database.	41
4.4	The ELTE IK Assistant web interface.	42

List of Tables

3.1	Minimum system requirements.	12
4.1	Runtime stores (above double line) and data pipeline artifacts (below double line) used by the ELTE IK Assistant.	24
4.2	Development and test machine specifications.	25
4.3	Project directory structure and file responsibilities.	27
4.4	Key configuration parameters in <code>app/settings.py</code>	28
4.5	API endpoints exposed by the FastAPI backend.	38
5.1	End-to-end test cases and their assertions.	45
5.2	The four evaluation configurations.	46
5.3	Evaluation results averaged over 20 test questions. Hit-rate is computed over in-scope questions only (15 questions); refusal rate is computed over out-of-scope questions only (5 questions).	48
5.4	Response latency in seconds. The non-functional requirement NF-4 sets a target of 30 seconds.	50
A.1	Full evaluation test set.	57

List of Algorithms

1 BFS Web Crawler 31

List of Codes

3.1	Expected terminal output during first-run setup.	13
3.2	Expected terminal output during first-run setup.	15
4.1	Central configuration (<code>app/settings.py</code>)	27
4.2	Text cleaning and unified file loader (<code>scripts/build_index.py</code>) . . .	29
4.3	Chunking configuration (<code>scripts/build_index.py</code>)	31
4.4	Incremental diff logic (<code>scripts/build_index.py</code>)	32
4.5	Embedding pipeline (<code>scripts/build_index.py</code>)	33
4.6	Collection creation and upsert (<code>scripts/build_index.py</code>)	33
4.7	Singletons and retrieval function (<code>app/rag.py</code>)	35
4.8	Prompt template (<code>app/rag.py</code>)	35
4.9	Ollama invocation (<code>app/rag.py</code>)	36
4.10	End-to-end RAG query (<code>app/rag.py</code>)	37
4.11	Chat endpoint (<code>app/main.py</code>)	38
4.12	Upload endpoint (<code>app/main.py</code>)	39
5.1	Upload endpoint (<code>app/main.py</code>)	45